

KeepClip — dokumentacja backendu

Serwer HTTP w procesie aplikacji, baza danych, endpointy i procesy w tle

Wersja aplikacji 2.3.0 · gałąź `csharp` · stan na 17 września 2026 · .NET 10, C# 14

Ten dokument opisuje wszystko, co dzieje się „pod spodem” KeepClip: jak uruchamia się proces, jak wygląda baza danych, jakie endpointy udostępnia serwer i jakie usługi działają w tle. Jest pisany dla osoby, która nie zna aktualnego stanu aplikacji i ma ją dalej rozwijać. Interfejs użytkownika opisuje osobny dokument *Frontend*, a narzędzia zewnętrzne (ffmpeg, Whisper, Google Drive, instalator, testy) dokument *Infrastruktura*.

Spis treści

1. Czym jest KeepClip i jak działa jako proces
 - 1.1 Zadanie aplikacji
 - 1.2 Jeden proces, dwie role
 - 1.3 Technologie
2. Uruchomienie krok po kroku (Program.cs)
 - 2.1 Heartbeat i samozamykanie
3. Struktura katalogów i warstwy
4. Konfiguracja: Config, settings.json, zmienne środowiskowe
 - 4.1 Klasa Config
 - 4.2 Plik settings.json
 - 4.3 Zmienne środowiskowe
5. Baza danych
 - 5.1 Tabele
 - 5.2 Pomocnicze rozszerzenia (DbExtensions)
 - 5.3 Repozytoria
6. Konwencje API
7. Referencja endpointów
 - 7.1 System i konfiguracja (ConfigEndpoints)
 - 7.2 Statystyki (StatsEndpoints)
 - 7.3 Klipy (ClipEndpoints)
 - 7.4 Segmenty transkrypcji (SegmentEndpoints)
 - 7.5 Wyszukiwanie (SearchEndpoints)
 - 7.6 Foldery (FolderEndpoints)
 - 7.7 Transkrypcja zbiorcza (TranscribeEndpoints)
 - 7.8 Natychmiastowa powtórka (ReplayEndpoints)
 - 7.9 Chmura — Google Drive (CloudEndpoints)
 - 7.10 Media i odtwarzanie (MediaEndpoints)
 - 7.11 Aktualizacje (UpdateEndpoints)
 - 7.12 Samouczek (OnboardingEndpoints)
8. Serwisy i procesy w tle
 - 8.1 Scanner (Workers/Scanner.cs)

- 8.2 ThumbnailWorker
- 8.3 TranscribeWorker i TranscribeState
- 8.4 Transcriber (Whisper)
- 8.5 ClipService
- 8.6 PlaybackProxyService
- 8.7 CloudService
- 8.8 UpdateService, OnboardingService, SingleOperationProgress
- 8.9 ReplayService (bufor powtórek) — skrót
- 9. Wątki, blokady i stabilność
- 10. Jak rozszerzać backend
 - 10.1 Nowy endpoint
 - 10.2 Nowe ustawienie
 - 10.3 Nowa kolumna w bazie
 - 10.4 Zasady

1. Czym jest KeepClip i jak działa jako proces

1.1 Zadanie aplikacji

KeepClip to aplikacja desktopowa dla Windows dla graczy. Obserwuje folder, do którego NVIDIA ShadowPlay, AMD ReLive lub własny bufor powtórek KeepClip zapisują nagrania z gier. Każde nagranie trafia do lokalnej biblioteki, dostaje miniaturę, a po transkrypcji mową (Whisper) można je wyszukać po wypowiedzianych słowach. Klipy można przycinać i kompresować pod limit Discorda, grupować w foldery, oznaczać jako ulubione, naprawiać uszkodzone nagrania oraz przenosić na własny Dysk Google, żeby zwolnić miejsce na dysku.

1.2 Jeden proces, dwie role

Plik `KeepClip.exe` uruchamia w jednym procesie dwie rzeczy:

- **Serwer HTTP** — ASP.NET Core (Kestrel, minimal API) nasłuchujący wyłącznie na `127.0.0.1` i losowym wolnym porcie. Nie ma uwierzytelniania, bo serwer nigdy nie jest widoczny spoza komputera.
- **Okno** — formularz WinForms bez ramki systemowej, w którym kontrolka WebView2 (silnik Edge/Chromium) łąduje adres serwera. Interfejs to zwykłe pliki HTML/JS/CSS z katalogu `frontend/`.

```
KeepClip.exe
├─ wątek główny: ASP.NET Core / Kestrel → http://127.0.0.1:<losowy port>/
│   ├── /static/*      pliki z frontend/
│   ├── /api/*         endpointy JSON (ten dokument)
│   ├── /video, /thumb strumienie wideo i miniatury
│   └─ workery: skaner, miniatury, transkrypcja, bufor powtórek, proxy odtwarzania
└─ wątek UI (STA): WinForms ShellForm + WebView2
    ├── most JS ⇄ C# (okno, wybór folderu, przeciąganie)
    ├── ikona w zasobniku, globalny skrót klawiszowy, powiadomienie o powtórcie
    └─ zamknięcie okna → zatrzymanie serwera
```

Tryb **samego serwera** (argument `server-only` albo zmienna `KEEPCLIP_SERVER_ONLY=1`) uruchamia tylko serwer na stałym porcie 8770, bez okna i bez blokady pojedynczej instancji. Korzystają z niego testy integracyjne i praca deweloperska w zwykłej przeglądarce.

1.3 Technologie

Obszar	Technologia	Uwagi
Platforma	.NET 10, C# 14, <code>net10.0-windows</code>	Projekt typu <code>Microsoft.NET.Sdk.Web</code> z <code>OutputType=WinExe</code> i WinForms.
HTTP	ASP.NET Core minimal API	Bez kontrolerów; każdy plik w <code>Endpoints/</code> to metoda rozszerzająca <code>Map...Endpoints</code> .
Baza	SQLite przez <code>Microsoft.Data.Sqlite</code> + <code>SQLitePCLRaw.bundle_e_sqlite3</code>	Surowy SQL, bez ORM. FTS5 do wyszukiwania pełnotextowego.
Mowa → tekst	Whisper.net 1.9.1 (+ Runtime, + Runtime.Vulkan)	Modele GGML pobierane do <code>models/</code> przy pierwszym użyciu.

Wideo	ffmpeg / ffprobe w <code>tools/bin</code>	Uruchamiane jako procesy potomne; nie ma bibliotek ffmpeg w procesie.
Dźwięk	NAudio 2.2.1 (WASAPI)	Miks wyjść i mikrofonu dla bufora powtórek.
Okno	WinForms + Microsoft.Web.WebView2	Pakiet WebView2 ciągnie też WPF, który jest usuwany celem <code>DropWebView2Wpf</code> w csproj.
Chmura	Google Drive API v3 + OAuth 2.0 PKCE	Własny klient HTTP, bez SDK Google.

2. Uruchomienie krok po kroku (Program.cs)

Plik `Program.cs` używa instrukcji najwyższego poziomu. Kolejność ma znaczenie, dlatego opisana jest krok po kroku.

1. `CrashLog.Install()` — jeszcze przed zbudowaniem hosta rejestruje obsługę `AppDomain.UnhandledException` i `TaskScheduler.UnobservedTaskException`. Każdy nieobsłużony wyjątek trafia do `data/crash.log`. W wydaniu publicznym nie ma konsoli ani panelu logów, więc to jedyny ślad po awarii.
2. **Tryb pracy** — `serverOnly` jest prawdą przy argumencie `server-only` lub zmiennej `KEEPCLIP_SERVER_ONLY=1`.
3. **Pojedyncza instancja** — w trybie okienkowym tworzony jest mutex `Local\KeepClip-SingleInstance`. Jeśli już istnieje, nowy proces wywołuje `DesktopShell.FocusExistingInstance()` (sygnalizuje zdarzenie `Local\KeepClip-ShowRequest`, a w ostateczności szuka okna po nazwie procesu) i kończy się. Dwie instancje walczyłyby o mechanizm Desktop Duplication i psuły bufor powtórek.
4. `DevLog.Install()` — w buildach ze stałą `KEEPCLIP_DEV` przechwytuje `Console.Error` do bufora kołowego (4000 wierszy) czytane przez panel logów w interfejsie. W wydaniu publicznym to pusta metoda.
5. **Filtry logów** — logi hostingu i routingu ASP.NET są podniesione do poziomu Warning, bo interfejs odpytuje status co kilka sekund.
6. **Adres** — tryb okienkowy: `http://127.0.0.1:{wolny port}` (port z `TcpListener` na porcie 0). Tryb serwera: konfiguracja `urls` albo `http://127.0.0.1:8770`.
7. **Opcje JSON** — `PropertyNamingPolicy = null`, `DictionaryKeyPolicy = null`, `DefaultIgnoreCondition = Never`. Nazwy pól są wysyłane dokładnie tak, jak zapisano je w kodzie (snake_case), a wartości `null` nie są pomijane.
8. **Rejestracja usług** — singletony: `ClipRepository`, `SegmentRepository`, `FolderRepository`, `StatsRepository`, `PlaybackProxyService`, `ClipService`. Reszta serwisów to klasy statyczne.
9. `Db.InitDb()` i katalogi `data/thumbs`, `data/tmp`, `data/playback`.
10. **Pliki statyczne** — `frontend/` pod ścieżką `/static`, z `ServeUnknownFileTypes` (ikony SVG, dźwięki WAV).
11. **Mapowanie endpointów** w kolejności: Config, Stats, Clip, Segment, Folder, Search, Transcribe, Replay, Cloud, Media, Update, Onboarding.
12. `StartIdleWatcher()` — wątek tła, który co 15 s sprawdza, czy interfejs daje znak życia (patrz *Heartbeat*). Wyłącza go zmienna `KEEPCLIP_IDLE_EXIT=0`.
13. `ReplayService.ApplyConfig()` — jeśli w ustawieniach bufor powtórek jest włączony, startuje ffmpeg. `ProcessExit` gwarantuje, że ffmpeg nie przeżyje procesu.
14. **Start** — tryb serwera: `app.Run()` blokuje do zamknięcia. Tryb okienkowy: `app.StartAsync()`, potem wątek `KeepClip-UI` (STA) z `DesktopShell.Run(baseUrl)`; po zamknięciu okna `ReplayService.Shutdown()` i `app.StopAsync()`.

Na końcu pliku jest deklaracja `public partial class Program { }`. Dzięki niej testy integracyjne mogą uruchomić cały program przez `WebApplicationFactory<Program>`.

2.1 Heartbeat i samozamykanie

Klasa `Heartbeat` trzyma znacznik czasu ostatniej aktywności interfejsu. Prawie każdy endpoint wywołuje `Heartbeat.Touch()`, a strona wysyła dodatkowo `POST /api/heartbeat` co 30 s. Wątek `KeepClip-Idle` kończy proces (`Environment.Exit(0)`), gdy cisza trwa dłużej niż 5 minut, chyba że trwa transkrypcja albo bufor powtórek pracuje w tle. Przy starcie obowiązuje 60 s karencji. Mechanizm istnieje po to, żeby proces nie został w pamięci, gdy okno `WebView2` zniknie w nietypowy sposób.

3. Struktura katalogów i warstwy

Katalog / plik	Rola
KeepClip/Program.cs	Start procesu, rejestracja usług, mapowanie endpointów.
KeepClip/Config.cs	Stałe ścieżki i parametry (odnajdywanie katalogu aplikacji, katalog danych, modele Whispera).
KeepClip/Settings.cs	Odczyt i zapis <code>data/settings.json</code> .
KeepClip/ReplayService.cs	Bufor natychmiastowej powtórki (ffmpeg + segmenty MPEG-TS).
KeepClip/TranscribeState.cs	Współdzielony stan transkrypcji zbiorczej.
KeepClip/DevLog.cs	Bufor logów dla panelu deweloperskiego (tylko <code>KEEPCLIP_DEV</code>).
KeepClip/GlobalUsings.cs	Globalne <code>using</code> dla wszystkich przestrzeni nazw projektu.
Endpoints/	Warstwa HTTP. Walidacja wejścia, tłumaczenie wyników na kody HTTP. Zero logiki domenowej poza prostymi przypadkami.
Services/	Logika: <code>ClipService</code> (naprawa, wycinanie, usuwanie, ponowna transkrypcja), <code>CloudService</code> , <code>UpdateService</code> , <code>PlaybackProxyService</code> , <code>OnboardingService</code> .
Data/	Repozytoria na surowym SQL: <code>Db</code> , <code>DbExtensions</code> , <code>ClipRepository</code> , <code>SegmentRepository</code> , <code>FolderRepository</code> , <code>StatsRepository</code> .
Domain/	Rekordy danych (<code>Clip</code> , <code>Segment</code> , <code>OnboardingState</code> , <code>Result<T></code> i inne).
Infrastructure/	Świat zewnętrzny: <code>ffmpeg</code> (<code>Media</code>), <code>Whisper</code> (<code>Transcriber</code>), <code>Google</code> (<code>OAuthService</code> , <code>GoogleDrive</code> , <code>CredentialStore</code>), okno (<code>DesktopShell</code> , <code>HotkeyManager</code> , <code>ReplayToast</code>), sprzęt (<code>GpuInfo</code> , <code>DisplayHelper</code> , <code>AudioPump</code>), <code>Trash</code> , <code>Strings</code> , <code>Heartbeat</code> , <code>CrashLog</code> , <code>SingleOperationProgress</code> .
Workers/	Procesy w tle: <code>Scanner</code> , <code>ThumbnailWorker</code> , <code>TranscribeWorker</code> .
frontend/	Interfejs (osobny dokument).
tests/	Projekt xUnit i skrypt kontroli tłumaczeń (dokument Infrastruktura).
installer/, <code>setup.ps1</code> , <code>tools/bin</code>	Budowanie instalatora, pobieranie ffmpeg.
<code>data/</code> , <code>models/</code>	Dane użytkownika i modele. Nie trafiają do gita. Wszystkie konfiguracje kompilacji dzielą ten sam katalog.

Reguła zależności: Endpoints → Services → Data. Infrastructure jest używane przez Services i Workers. Endpoints nie powinny wywoływać ffmpeg ani Google bezpośrednio, choć w kilku miejscach (np. `/thumb`, `/video`) robią to dla prostoty.

4. Konfiguracja: Config, settings.json, zmienne środowiskowe

4.1 Klasa Config

Pola statyczne, inicjalizowane przy pierwszym użyciu klasy. Z tego powodu zmienne środowiskowe muszą być ustawione, zanim cokolwiek dotknie Config (testy robią to w inicjalizatorze modułu).

Pole	Wartość
AppRoot	Pierwszy katalog w górę od <code>AppContext.BaseDirectory</code> , który zawiera <code>frontend/index.html</code> . Działa zarówno w instalacji (exe obok <code>frontend/</code>), jak i w repozytorium (<code>KeepClip/bin/Release/net10.0-windows/</code> → korzeń repo).
DataDir	<code>KEEPCLIP_DATA_DIR</code> , a gdy brak — <code>AppRoot/data</code> .
ThumbsDir, TmpDir, PlaybackDir	<code>data/thumbs</code> , <code>data/tmp</code> , <code>data/playback</code> .
DbPath	<code>data/kliпы.db</code> .
ToolsBin, Ffmpeg, Ffprobe	<code>AppRoot/tools/bin/ffmpeg.exe</code> i <code>ffprobe.exe</code> .
FrontendDir	<code>AppRoot/frontend</code> .
ModelsDir, WhisperModelPath(typ)	<code>AppRoot/models/ggml-{typ}.bin</code> .
DefaultClipsRoot	Systemowy folder Wideo użytkownika.
CutsSubdir	Wycinki — domyślny podfolder na wycięte fragmenty.
VideoExts	<code>.mp4</code> <code>.mkv</code> <code>.mov</code> <code>.avi</code> <code>.webm</code> (bez rozróżniania wielkości liter).
GoogleClientPath	<code>data/google_client.json</code> — klient OAuth wbudowany przez instalator.
DriveFolderName	<code>KeepClip</code> — folder na Dysku Google.
DriveTokenTarget	<code>KeepClip:GoogleDriveRefreshToken</code> — nazwa wpisu w Menedżerze poświadczeń Windows.
UiLanguages, DefaultLanguage	<code>pl</code> <code>en</code> <code>ru</code> <code>uk</code> ; domyślnie <code>en</code> (aplikacja trafia do wielu krajów).
WhisperModelLadder	<code>large-v3</code> → <code>large-v3-turbo</code> → <code>small</code> .
WhisperModelMinVram	<code>large-v3</code> : 6 GB, <code>large-v3-turbo</code> : 3 GB, <code>small</code> : 0. Zmierzone zużycie to ok. 4,3 GB i 2,3 GB; progi zostawiają zapas na grę w tle.
WhisperModelOverride	Z <code>KEEPCLIP_WHISPER_MODEL</code> ; wyłącza automatyczny dobór modelu.
WhisperSplitGapSeconds	1,5 s — cisza, po której wypowiedź jest dzielona na osobne segmenty.

4.2 Plik settings.json

Płaski obiekt JSON w `data/settings.json`. Klasa `Settings` czyta go przy każdym wywołaniu (bez pamięci podręcznej) i zapisuje pod blokadą. Uszkodzony plik jest traktowany jak pusty, a pierwszy zapis go naprawia.

Klucz	Typ	Znaczenie
<code>clips_root</code>	string	Folder biblioteki. Ustawienie go ustawia też <code>configured=true</code> .
<code>configured</code>	bool	Czy świeża instalacja przeszła wybór folderu. <code>GET /api/config</code> ustawia go także wtedy, gdy baza ma już klipy (stare instalacje).
<code>cuts_root</code>	string	Folder wycinków; domyślnie <code>clips_root/Wycinki</code> .
<code>ui_language</code>	string	Język interfejsu i transkrypcji. Zapisany po stronie serwera, bo transkrypcja działa bez okna.
<code>accent_theme</code>	<code>purple</code> <code>green</code>	Kolor akcentów.
<code>window_state</code>	<code>max;x;y;w;h</code>	Położenie okna. Przywracane tylko, gdy przecina podłączony ekran.
<code>replay_enabled</code> , <code>replay_background</code> , <code>replay_mic</code>	<code>"1"</code> / <code>"0"</code>	Bufor powtórek: włączony, praca w zasobniku po zamknięciu okna, mikrofon.
<code>replay_duration_s</code> , <code>replay_fps</code> , <code>replay_quality</code> , <code>replay_hotkey</code>	string	15–600 s (domyślnie 120), 30/60/90 (60), low/medium/high (high), skrót (Alt+F10).
<code>replay_audio_output</code> , <code>replay_audio_input</code>	string	Identyfikatory urządzeń WASAPI; pusty = domyślne / wszystkie wyjścia.
<code>whisper_model</code>	string	Ostatnio załadowany model.
<code>whisper_model_probe</code>	string	Znacznik „właśnie ładuję model X”. Jeśli po restarcie nadal tam jest, poprzedni proces zginął przy ładowaniu (brak pamięci GPU) i model trafia do <code>whisper_model_blocked</code> .
<code>whisper_model_blocked</code>	string	Model wyłączony na tym komputerze.
<code>cloud_folder_id</code>	string	Identyfikator folderu KeepClip na Dysku Google.
<code>onboarding</code>	obiekt	Stan samouczka: <code>started</code> , <code>completed</code> , <code>skipped</code> , <code>current_step</code> , <code>completed_version</code> , <code>seen_hints[]</code> .

4.3 Zmienne środowiskowe

Zmienna	Działanie
<code>KEEPCLIP_DATA_DIR</code>	Osobny katalog danych. Testy ustawiają go na katalog w Temp, żeby nie dotykać biblioteki użytkownika.
<code>KEEPCLIP_SERVER_ONLY=1</code>	Tylko serwer na porcie 8770, bez okna i muteksu.
<code>KEEPCLIP_IDLE_EXIT=0</code>	Wyłącza samozamykanie po 5 minutach ciszy.
<code>KEEPCLIP_WHISPER_MODEL</code>	Wymusza model transkrypcji (np. <code>small</code>).

KEEPCLIP_CAPTURE_GDI=1	Wymusza przechwytywanie ekranu przez GDI zamiast Desktop Duplication.
------------------------	---

Stała kompilacji `KEEPCLIP_DEV` jest zdefiniowana we **wszystkich** konfiguracjach poza `-p:PublicRelease=true`. Lokalny `-c Release` nadal ma panel logów i endpoint `/api/logs`. Build taki, jaki dostają użytkownicy, robi `installer\build.ps1 -PublicRelease`.

5. Baza danych

Jedna baza SQLite `data/klipy.db`. Schemat tworzy `Db.InitDb()` (idempotentne `CREATE ... IF NOT EXISTS`), a metoda `Migrate` dokłada kolumny, których nie ma w starszych bazach (`favorite`, `storage`, `remote_id`, `remote_uploaded_at`). Każde otwarcie połączenia ustawia `ForeignKeys=true`, `DefaultTimeout=30` i `PRAGMA busy_timeout=30000`, więc równoległe zapisy z workerów czekają zamiast rzucać błąd.

5.1 Tabele

clips

Kolumna	Znaczenie
<code>id</code>	Klucz główny, AUTOINCREMENT (identyfikatory nie są ponownie używane).
<code>game</code>	Nazwa gry = nazwa podfolderu, w którym leży plik; <code>_root</code> dla plików bezpośrednio w folderze biblioteki.
<code>filename</code> , <code>filepath</code>	Nazwa i pełna ścieżka. <code>filepath</code> jest unikalna.
<code>size_bytes</code> , <code>mtime</code>	Rozmiar i czas modyfikacji w sekundach Unix (z ułamkiem).
<code>duration</code>	Czas trwania w sekundach; NULL, dopóki ffmpeg go nie zmierzy.
<code>has_thumb</code>	0 = brak / do zrobienia, 1 = jest, 2 = nie udało się (nie ponawiać, dopóki skan nie wyzeruje).
<code>transcribed_at</code> , <code>language</code>	Znacznik ISO 8601 i język ostatniej transkrypcji.
<code>favorite</code>	0/1.
<code>storage</code>	<code>local</code> albo <code>cloud</code> . Klip w chmurze nie ma pliku na dysku, ale zachowuje <code>filepath</code> jako zarezerwowaną ścieżkę docelową.
<code>remote_id</code> , <code>remote_uploaded_at</code>	Identyfikator pliku na Dysku Google i czas wysłania.

segments

Wersy transkrypcji: `id`, `clip_id` (kaskadowe usuwanie), `start_s`, `end_s`, `text`. Indeks po `clip_id`.

segments_fts

Wirtualna tabela FTS5 z zewnętrzną treścią (`content='segments'`). Trzy wyzwalacze (`segments_ai`, `segments_ad`, `segments_au`) utrzymują indeks przy INSERT/DELETE/UPDATE. Tokenizer `unicode61_remove_diacritics 2` sprawia, że „gesła” znajduje „gęślą”. Uwaga: litera `ł` nie jest w Unicode znakiem z diakrytykiem, więc „zazolc” nie znajdzie „zażółć” przez `ł`.

folders, folder_clips

Foldery użytkownika (nazwa unikalna, `created_at`) i tabela łącząca z kluczem złożonym (`folder_id`, `clip_id`), obie relacje kaskadowe. Indeksy po `clip_id` i po (`folder_id`, `added_at DESC`).

5.2 Pomocnicze rozszerzenia (DbExtensions)

Wszystkie repozytoria zwracają `Dictionary<string, object?>` zamiast klas, żeby JSON miał dokładnie te nazwy kolumn, których oczekuje interfejs. `DBNull` jest zamieniane na `null`. Metody: `Query` (lista wierszy), `QueryOne`, `Scalar`, `ScalarLong`, `Exec`. Parametry przekazuje się jako krotki (`"$nazwa"`, wartość).

5.3 Repozytoria

Klasa	Metody i zachowanie
<code>ClipRepository</code>	<code>List(game, sort, limit, favoritesOnly)</code> — sortowania <code>newest</code> (domyślne), <code>oldest</code> , <code>largest</code> , <code>smallest</code> ; nieznane wraca do <code>newest</code> . <code>GetRow</code> (wszystkie kolumny), <code>GetById</code> (rekord <code>Clip</code>), <code>Count</code> , <code>ToggleFavorite</code> (null gdy brak klipu), <code>Insert</code> , <code>UpdateAfterFix</code> , <code>SetTranscribed</code> , <code>SetThumbState</code> , <code>Delete</code> .
<code>SegmentRepository</code>	<code>ListByClipId</code> (posortowane po <code>start_s</code>), <code>UpdateText</code> , <code>DeleteByClipId</code> , <code>Insert</code> , <code>ReplaceAll</code> (w transakcji), <code>ShiftTimestamps(clipId, offset)</code> — po przycięciu początku klipu przesuwają czasy, kasuje segmenty kończące się przed zerem i przycina start do 0. <code>Search(ftsQuery, sort, limit)</code> — zapytanie FTS z <code>bm25</code> , fragment <code>snippet(..., '<mark>', '</mark>', '...', 12)</code> ; sortowania: <code>relevance</code> , <code>newest</code> , <code>oldest</code> , <code>largest</code> , <code>smallest</code> .
<code>FolderRepository</code>	<code>List</code> (z <code>clip_count</code> i do 4 ostatnio dodanych <code>sample_clip_ids</code> , sortowanie po nazwie bez wielkości liter), <code>Create</code> (null przy duplikacie — kod SQLite 19), <code>Rename</code> → <code>FolderWrite {Ok, NotFound, Duplicate}</code> , <code>Delete</code> (zwraca nazwę albo null), <code>ListClips</code> (sortowania <code>newest</code> , <code>oldest</code> , <code>largest</code> , <code>smallest</code> , <code>added</code>), <code>AddClip</code> → <code>AddClipResult {Ok, NoFolder, NoClip}</code> (INSERT OR IGNORE), <code>RemoveClip</code> , <code>GetFolderIdsForClip</code> .
<code>StatsRepository</code>	<code>Get(diskTotal, diskFree)</code> — liczby klipów, transkrypcji, segmentów, bajty łącznie / lokalne / w chmurze, ulubione, lista gier z liczbą klipów i transkrypcji.

6. Konwencje API

- **Adres bazowy:** `http://127.0.0.1:<port>`. Port zna tylko okno WebView2 (i `netstat -ano` po PID procesu).
- **Format:** JSON w obie strony; ciała żądań to rekordy C# z nazwami pól w snake_case (`clips_root`, `target_size_mb`). Wiązanie nie rozróżnia wielkości liter.
- **Błędy:** `Api.Detail(kod, komunikat, error?)` zwraca `{ "detail": "tekst w języku użytkownika", "error": "kodBledu" | null }`. Interfejs najpierw próbuje przetłumaczyć `errors.<kod>` we własnym słowniku, a gdy klucza nie ma, pokazuje `detail`. Nowy błąd powinien dostać kod, jeśli interfejs ma go tłumaczyć inaczej niż serwer.
- **Teksty serwera:** klasa `Strings` z 56 kluczami w czterech językach (język z `Settings.GetLanguage()`, zapas `en`, brak klucza zwraca sam klucz). Argumenty przez `string.Format`. Testy pilnują, żeby każdy klucz użyty w kodzie istniał we wszystkich językach.
- **Heartbeat:** endpointy wywoływane przez użytkownika robią `Heartbeat.Touch()`; czysto statusowe (`/api/stats`, `/api/transcribe/status`) nie zawsze.
- **Logowanie:** `DevLog.Add` po ważnych akcjach; działa tylko w buildzie deweloperskim.
- **Bezpieczeństwo:** brak uwierzytelniania i CORS — serwer nasłuchuje wyłącznie na pętli zwrotnej. Ścieżki plików z żądań są używane tylko po sprawdzeniu istnienia (`/api/config`, `/api/show-in-explorer`).

Kod error	Gdzie	HTTP
<code>clipNotFound</code> , <code>transcriptEmpty</code>	<code>transcript.txt</code>	404
<code>folderDuplicate</code> , <code>folderNameTooLong</code>	foldery	409, 400
<code>segmentTextEmpty</code>	edycja segmentu	422
<code>configPathRequired</code> , <code>configBadTheme</code> , <code>configBadLanguage</code> , <code>configTranscribeBusy</code>	konfiguracja	400, 400, 400, 409
<code>cloudNotConnected</code> , <code>cloudExpired</code>	chmura	400, 401
<code>replayBadDuration</code> , <code>replayBadFps</code> , <code>replayBadQuality</code> , <code>replayBadHotkey</code>	powtórki	400
<code>updateInProgress</code>	aktualizacja	409

7. Referencja endpointów

Poniżej wszystkie 57 tras zmapowanych w `Program.cs`, pogrupowane jak pliki w `Endpoints/`. Przy każdym: co robi, parametry, odpowiedź, błędy, skutki uboczne i kto z niego korzysta w interfejsie.

7.1 System i konfiguracja (`ConfigEndpoints`)

GET /

Serwuje `frontend/index.html`. Przed wysłaniem: dopisuje `data-accent-theme="purple|green"` do `<html>`, dla motywu zielonego podmienia favicon, a do odnośników `/static/styles.css`, `app.js`, `onboarding.css`, `onboarding.js` dokleja `?v=<czas modyfikacji>`, żeby przeglądarka nie użyła starej wersji po aktualizacji. Brak pliku → HTML z kodem 500.

GET /api/ping

Odpowiada `{"ok":true}`. Używany przez testy i diagnostykę.

GET /api/config

Bieżąca konfiguracja. Skutek uboczny: gdy `configured` jest fałszem, a baza ma klipy (stare instalacje sprzed pliku ustawień), zapisuje `clips_root` i oznacza konfigurację jako gotową.

```
{ "clips_root": "F:\\\\NVIDIA", "clips_root_exists": true, "configured": true,
  "dev": true, "accent_theme": "green", "language": "pl", "version": "2.3.0" }
```

Kto używa: start strony (`bootstrap`), modal wyboru folderu, panel logów (pole `dev`), samouczek (pole `configured`).

POST /api/config

Ustawia folder biblioteki. Ciało: `{"clips_root": "C:\\\\..."}`. Ścieżka jest przycinana z białych znaków i cudzysłowów (użytkownicy wklejają ścieżkę z Eksploratora).

Warunek	Odpowiedź
trwa transkrypcja	409 <code>configTranscribeBusy</code>
pusta ścieżka	400 <code>configPathRequired</code>
ścieżka nie istnieje / to plik	400 (tekst z nazwą ścieżki)
OK	<code>{"ok":true,"clips_root":"...", "scan":{"found,added,removed,updated}}</code>

Skutki: zapis ustawień, natychmiastowy skan (`Scanner.Scan`) i start workera miniatur.

POST /api/accent-theme

Ciało `{"theme":"purple"|"green"}`. Inna wartość → 400 `configBadTheme`. Odpowiedź `{"ok":true,"theme":"green"}`. Motyw wpływa na `GET /`.

POST /api/language

Ciało `{"language":"pl"}` (wielkość liter nieistotna). Spoza `pl en ru uk` → 400 `configBadLanguage`. Interfejs zgłasza tu język przy starcie i przy zmianie w ustawieniach, bo od tego zależy język rozpoznawania mowy.

POST /api/heartbeat

Sygnał życia od strony (co 30 s). Odpowiedź `{"ok":true}`.

POST /api/scan

Uruchamia skan folderu biblioteki i workera miniatur. Odpowiedź: `{"found":n,"added":n,"removed":n,"updated":n}`, a przy braku folderu dodatkowo `"error"`. Interfejs woła go przy starcie, co 2,5 s w tle (auto-sync), przed transkrypcją i przyciskiem „Skanuj”.

POST /api/shutdown

Kończy proces po 500 ms (żeby odpowiedź zdążyła wyjść). Podczas transkrypcji odpowiada `{"ok":false,"reason":"transcription_running"}` i nie zamyka.

GET /api/logs?since=N • **POST** /api/logs/clear

Tylko w buildzie z KEEPCLIP_DEV (blok #if). Zwraca {"seq":ostatniNumer,"lines":[...]} z wpisami nowszymi niż since. Panel logów odpytuje co 1,5 s.

7.2 Statystyki (StatsEndpoints)

GET /api/stats

Liczby dla paska bocznego i filtra gier. Miejsce na dysku bierze z DriveInfo dysku, na którym leży folder biblioteki (błąd → null).

```
{ "clips": 235, "transcribed": 74, "segments": 5120, "total_bytes": ..., "local_bytes": ...,  
  "cloud_bytes": ..., "disk_total": ..., "disk_free": ..., "favorites": 3,  
  "games": [ { "game": "Counter-Strike 2", "clips": 120, "done": 60 }, ... ] }
```

7.3 Klipy (ClipEndpoints)

GET /api/clips?game=&sort=&limit=&favorite=

Lista klipów. sort: newest (domyślnie), oldest, largest, smallest. limit domyślnie 200. favorite=1 zawęża do ulubionych. Każdy wiersz: id, game, filename, duration, size_bytes, mtime, has_thumb, transcribed_at, favorite, storage. Widok Klipy pobiera 120 najnowszych, widok Ulubione 500.

GET /api/single-operation/status

Postęp jednej operacji na klipie (wycinanie albo ponowna transkrypcja): {id, kind: "cut"|"transcription", stage, clip_id, progress 0-1, active, success, error}. Interfejs odpytuje co 300 ms podczas operacji. Jednocześnie śledzona jest tylko jedna operacja (SingleOperationProgress).

POST /api/clips/{id}/favorite

Przełącza gwiazdkę. Odpowiedź {"ok":true,"id":...,"favorite":true|false}; 404 gdy brak klipu.

GET / **HEAD** /api/clips/{id}/transcript.txt

Transkrypcja jako plik tekstowy do pobrania. Nagłówek `Content-Disposition: attachment; filename="<nazwa klipu>.txt"` sprawia, że WebView2 zapisuje plik zamiast go wyświetlać. Treść: nazwa pliku, wiersz `gra · data`, pusty wiersz, potem `[mm:ss]` tekst (lub `[h:mm:ss]` po godzinie). Zaczyna się znacznikiem BOM, żeby Notatnik pokazał polskie znaki (`getBytes` sam go nie dodaje, więc jest doklejany z `GetPreamble`). HEAD pozwala interfejsowi sprawdzić dostępność bez pobierania. Błędy: 404 `clipNotFound`, 404 `transcriptEmpty`.

POST /api/clips/{id}/retranscribe

Transkrybuje jeden klip od nowa (synchronicznie — żądanie trwa tyle, ile transkrypcja). Postęp raportuje przez `SingleOperationProgress`. Odpowiedź `{"ok":true,"segments":n,"seconds":s}`. Błędy: 409 gdy trwa transkrypcja zbiorcza, 404 brak klipu lub pliku, 500 błąd Whispera. Zapisuje segmenty, `transcribed_at` i język.

POST /api/clips/{id}/fix

Naprawa klipu z uszkodzonym początkiem (typowe dla NVIDIA ShadowPlay): kolejno remuks bez zmian, potem pominięcie 1, 3, 7, 15, 25, 35, 60 s, aż `ffprobe` przestanie zgłaszać błędy dekodowania w pierwszych 3 s. Oryginał idzie do Kosza Windows, naprawiony plik wchodzi na jego miejsce z zachowaną datą modyfikacji, miniatura jest robiona od nowa, segmenty przesuwane o przycięty czas, proxy odtwarzania unieważniane. Odpowiedź: `{ok, message, trimmed_seconds, new_duration, new_size_bytes, segments_shifted, segments_dropped}`. Błędy 404/422/500.

POST /api/clips/{id}/cut

Wycina fragment. Ciało `{"start": s, "end": s, "target_size_mb": null | liczba}`. Plik wynikowy `<nazwa>_cut-<start>-<koniec>[_<MB>MB].mp4` trafia do folderu wycinków (kolizje dostają `_2`, `_3`...). Jeśli folder wycinków leży wewnątrz biblioteki, wycinek jest od razu dodawany do bazy (`clip_id`, `in_library`). Ograniczenia: długość 0,1 s – 30 min. Przy limicie rozmiaru serwer liczy przepływność wideo z budżetu (limit – 100 kB marginesu – audio 128 kb/s – 50 kB kontenera), koduje NVENC H.264 i przy przekroczeniu limitu ponawia do 3 razy z obniżoną przepływnością. Bez limitu: stała jakość `cq 19`. Odpowiedź zawiera `output_path`, `output_directory`, `output_name`, `cuts_root`, `clip_id`, `in_library`, `duration_s`, `size_bytes`, `size_mb`.

Wycinanie wymaga enkodera `h264_nvenc` (karta NVIDIA). Na innych kartach `ffmpeg` zwróci błąd 422 z ostatnią linią `stderr`.

DELETE /api/clips/{id}?delete_file=true|false

Usuwa klip z bazy (kaskadowo segmenty i powiązania z folderami), miniaturę i proxy. Z `delete_file=true` (domyślnie) plik lokalny idzie do Kosza z ponawianiem (6 prób co 250 ms, bo odtwarzacz może jeszcze trzymać uchwyt), a klip w chmurze jest przenoszony do kosza Dysku Google. Odpowiedź `{deleted_clip_id, filename, file_sent_to_trash, cloud_deleted, trash_error}`. Błędy: 404, 409 (klip w chmurze bez `remote_id`), 502 (błąd Google).

GET /thumb/{id}

Miniatura JPEG (1280 px szerokości, klatka z 5. sekundy albo z połowy krótszego klipu). Generowana leniwie przy pierwszym żądaniu; nieudana próba zapisuje `has_thumb=2`, żeby nie ponawiać przy każdym przewinięciu listy. Skan zeruje ten znacznik. 404, gdy klipu nie ma albo ffmpeg nie dał rady.

7.4 Segmenty transkrypcji (SegmentEndpoints)

GET /api/segments/{clipId}

Otwarcie odtwarzacza: `{"clip": {wszystkie kolumny klipu}, "segments": [{id, start_s, end_s, text}, ...]}`. 404 gdy brak klipu.

PATCH /api/segments/{segmentId}

Ręczna poprawka wersu. Ciało `{"text": "..."} (przycinane)`. Pusty tekst → 422 `segmentTextEmpty`; brak segmentu → 404. Wyzwalacz FTS aktualizuje indeks wyszukiwania. Odpowiedź `{"ok": true, "id": ..., "text": "..."}.`

7.5 Wyszukiwanie (SearchEndpoints)

GET /api/search?q=&sort=&limit=

Wyszukiwanie po słowach z transkrypcji. Brak parametru `q` → 400 (wiązanie), pusty `q` → 422. Zapytanie użytkownika jest tłumaczone na składnię FTS5 metodą `ToFtsQuery`: każde słowo (litery, cyfry, podkreślenie, znaki łacińskie z diakrytykami) staje się tokenem prefiksowym w cudzysłowie, a tokeny łączy `AND`: `"headshot"* AND "smoke"*`. Dzięki cudzysłowom słowa `OR`, `NOT`, `NEAR` nie działają jak operatory. Same znaki interpunkcyjne dają pustą listę wyników. Odpowiedź: `{"query": q, "fts": "...", "results": [{segment_id, start_s, end_s, text, clip_id, game, filename, duration, size_bytes, mtime, has_thumb, favorite, storage, snippet, score}]}`. `snippet` zawiera znaczniki `<mark>`; interfejs koduje resztę HTML. Domyślny `limit` 100, interfejs prosi o 200 i filtruje po grze po swojej stronie.

7.6 Foldery (FolderEndpoints)

GET /api/folders

Lista folderów: [{id, name, created_at, clip_count, sample_clip_ids:[...do 4]}].

POST /api/folders

Ciało {"name":"..."} . Pusta nazwa → 400, ponad 100 znaków → 400 folderNameTooLong, duplikat → 409 folderDuplicate. Odpowiedź: wiersz {id, name, created_at}.

PATCH /api/folders/{id}

Zmiana nazwy; 404 / 409 jak wyżej; odpowiedź {"ok":true,"id":...,"name":"..."}.

DELETE /api/folders/{id}

Usuwa folder i powiązania (klipy zostają). Odpowiedź {"ok":true,"deleted_name":"..."}, 404 gdy nie istnieje.

GET /api/folders/{id}/clips?sort=&limit=

{"folder": {id,name,created_at}, "clips": [...]}. Sortowania jak w repozytorium; interfejs używa added (ostatnio dodane pierwsze). Domyślny limit 500.

POST / **DELETE** /api/folders/{folderId}/clips/{clipId}

Dodaje (idempotentnie) lub usuwa klip z folderu. 404 przy braku folderu lub klipu. Odpowiedź {"ok":true}.

GET /api/clips/{clipId}/folders

{"folder_ids":[...]} — foldery, w których jest klip (pusta lista dla nieznanego klipu).

7.7 Transkrypcja zbiorcza (TranscribeEndpoints)

GET /api/transcribe/status

Migawka TranscribeState: {running, total, done, current: {id, filename, game} | null, error, finished_at, log_tail: [ostatnie 8 wpisów]}.

POST /api/transcribe/start?force=true|false

Startuje worker dla klipów bez transkrypcji (albo wszystkich przy force), od najnowszych. Odpowiedź {"started":true,"force":..., "total":n} albo {"started":false,"reason":"already_running"}. Lista klipów jest liczona synchronicznie, żeby odpowiedź znała dokładną liczbę.

POST /api/transcribe/cancel

Ustawia flagę anulowania; worker zatrzymuje się po bieżącym klipie. {"cancelled":true} albo {"cancelled":false,"reason":"not_running"}.

GET /api/transcribe/stream

Server-Sent Events (text/event-stream). Co 500 ms serializuje migawkę i wysyła ramkę data: {...} tylko wtedy, gdy zmieniła się względem poprzedniej. Po zakończeniu (running=false i finished_at) wysyła stan końcowy i zamyka strumień. Własny serializator zachowuje snake_case, bo ręczny zapis omija opcje JSON z DI.

7.8 Natychmiastowa powtórka (ReplayEndpoints)

GET /api/replay/status

Celowo bez blokady, żeby działał także podczas problemów z ffmpeg: {enabled, background, running, saving, duration_s, fps, quality, hotkey, hotkey_active, encoder, capture, audio, mic_enabled, mic_active, audio_output, audio_input, error}. Interfejs odpytuje co 15 s stale i co 3 s przy otwartym oknie ustawień powtórki.

POST /api/replay/config

Ciało (wszystkie pola opcjonalne): {enabled, background, duration_s 15-600, fps 30|60|90, quality low|medium|high, hotkey "Alt+F10", mic, audio_output, audio_input}. Walidacja → 400 z kodami replayBad*; skrót jest sprawdzany przez HotkeyManager.TryParse (co najmniej jeden modyfikator + jeden klawisz). Puste audio_* oznaczają powrót do domyślnych. Po zapisie:

ReplayService.ApplyConfig() (start/stop/restart bufora, gdy zmienił się podpis ustawień) i HotkeyManager.Refresh(). Odpowiedź: status jak wyżej.

POST /api/replay/save

Zapisuje ostatnie N sekund bufora do <biblioteka>/<gra>/<gra> yyyy-MM-dd HH-mm-ss.mp4.

Odpowiedź {"ok":true,"file":...,"game":...,"path":...}; 409 gdy bufor nie działa lub zapis trwa; 500 przy błędzie łączenia. Ten sam kod uruchamia globalny skrót klawiszowy.

GET /api/replay/audio-devices

{"outputs":[{"id,name}], "inputs":[{"id,name}]}

 z NAudio (aktywne urządzenia WASAPI).

7.9 Chmura — Google Drive (CloudEndpoints)

GET /api/cloud/status

{configured, connected, name, email, photo, usage, limit, error}. configured = istnieje data/google_client.json; connected = jest token odświeżania w Menedżerze poświadczeń. Dane konta z pamięci podręcznej 20 s. Błąd przejściowy zostawia connected=true z polem error, żeby panel nie migał; cofnięta zgoda (invalid_grant) rozłącza lokalnie.

GET /api/cloud/avatar

Zdjęcie profilowe pobrane przez serwer (tylko z domen googleusercontent.com / ggph.com, do 1 MB, typ image/*). Cache-Control: private, max-age=300. 404 przy braku.

POST /api/cloud/connect

Rozpoczyna OAuth PKCE: uruchamia lokalny HttpListener na losowym porcie, zwraca {"auth_url":"https://accounts.google.com/..."}, który interfejs otwiera w przeglądarce systemowej. Zgoda ważna 5 minut. 400 gdy brak pliku klienta.

POST /api/cloud/disconnect

Usuwa token lokalnie i unieważnia go u Google. {"ok":true}.

POST /api/cloud/sync

Pobiera listę plików z folderu KeepClip na Dysku i uzgadnia z bazą (import brakujących, aktualizacja istniejących). Odpowiedź {imported, updated, found, skipped}. 400 cloudNotConnected, 401 cloudExpired, 500.

POST /api/clips/{id}/upload • /api/clips/{id}/download

Wysyła klip do chmury (plik lokalny idzie do Kosza) albo ściąga go z powrotem (plik na Dysku idzie do kosza Google). {"ok":true,"already":bool} — already, gdy klip był już w docelowym stanie. 404 brak pliku, 401 cloudExpired, 500.

POST /api/folders/{id}/upload

Wysyła wszystkie klipy folderu: {uploaded, total, skipped, failed}.

7.10 Media i odtwarzanie (MediaEndpoints)

GET /api/playback/{id}?start=true|false

Pyta, jak odtworzyć klip. Klipy do 120 FPS i klipy w chmurze dostają od razu {"ready":true,"url":"/video/{id}?v=rozmiar","proxy":false,"source_fps":...}. Klipy powyżej 120 FPS (WebView2 nie dekoduje ich płynnie) dostają jednorazową kopię podglądową 1080p60 generowaną w tle: odpowiedź {ready:false, preparing:true, progress, stage:"preparing"|"converting"}, a po zakończeniu {ready:true, url":"/playback/{id}?v=<odcisk>", proxy:true}. start=false tylko sprawdza, bez uruchamiania konwersji (podgląd po najejchaniu). Odcisk = rozmiar + czas modyfikacji pliku, więc zmiana oryginału unieważnia kopię.

GET /playback/{id}

Strumień gotowej kopii podglądowej (MP4, obsługa Range). 404 gdy kopia nie istnieje.

GET /video/{id}

Oryginalny plik z obsługą `Range` (przewijanie). Klip w chmurze jest przekazywany strumieniowo z Dysku Google z zachowaniem nagłówków `Range / Content-Range /206`. Typ MIME po rozszerzeniu.

POST /api/show-in-explorer

Ciało `{"path": "..."}` . Otwiera Eksplorator: dla pliku z `/select`, (zaznaczony plik), dla folderu sam folder. 404 gdy ścieżka nie istnieje.

7.11 Aktualizacje (UpdateEndpoints)

GET /api/update/check

Sprawdza najnowsze wydanie na GitHubie (`releases/latest`, limit 15 s). Wynik trzymany 6 h (sukces) albo 30 min (błąd). `{ok, current, latest, available, url, notes_url}`; `available` tylko gdy wersja jest nowsza i wydanie ma plik `KeepClip-Setup.exe`.

GET /api/update/status

`{phase: "idle"|"downloading"|"starting"|"error", percent, error}`.

POST /api/update/install

Pobiera instalator do `%TEMP%\KeepClip-Setup.exe`, uruchamia go i po 1,5 s kończy aplikację. Plik pobrany przez `HttpClient` nie ma znacznika Mark-of-the-Web, więc SmartScreen go nie blokuje. 409 `updateInProgress`, gdy instalacja już trwa.

7.12 Samouczek (OnboardingEndpoints)

GET /api/onboarding

Stan: `{started, completed, skipped, current_step, completed_version, current_version, seen_hints[]}`.

POST /api/onboarding

Częściowa zmiana: {started?, completed?, skipped?, current_step?, seen_hint?}. Ukończenie stempluje completed_version, kasuje skipped i current_step; pominięcie kasuje krok. Podpowiedzi są deduplikowane, lista ma limit 200 (najstarsze wypadają). Pusty/biały current_step nie zmienia poprzedniego. Niepoprawny JSON → 400.

POST /api/onboarding/restart

Ręczne uruchomienie z ustawień: started=true, skipped=false, current_step=null, ale completed zostaje.

8. Serwisy i procesy w tle

8.1 Scanner (Workers/Scanner.cs)

Uzgadnia bazę z dyskiem. Pod blokadą: (1) `Db.InitDb()`; (2) brak folderu → wynik z `error`; (3) przegląd rekurencyjny plików o dozwolonych rozszerzeniach; pliki zmodyfikowane w ostatnich 1,5 s są traktowane jako „jeszcze zapisywane” i pomijane do następnego skanu (liczą się w `found`, ale nie w `added`); (4) w jednej transakcji: `has_thumb=2` → `0` (ponowna szansa dla miniatur), klipy lokalne bez pliku są usuwane wraz z miniaturą (`removed`), klipy w chmurze zostają zawsze, zmiana rozmiaru/czasu istniejącego pliku zeruje miniaturę (`updated`), nowe pliki są wstawiane z `grą` = nazwa folderu nadrzędnego albo `_root`. Ścieżki zajęte przez klipy chmurowe nie tworzą duplikatów.

8.2 ThumbnailWorker

`Ensure()` uruchamia (jeśli nie działa) zadanie, które w pętli bierze pierwszy klip z `has_thumb=0` lub bez `duration`, mierzy czas `ffprobe` i robi miniaturę `ffmpeg`. Każdy identyfikator jest próbowany raz na przebieg (uszkodzony plik nie zapętla workera). Brak pliku → `has_thumb=2`. Bez `ffmpeg` każda próba kończy się szybko wartością 2.

8.3 TranscribeWorker i TranscribeState

`TranscribeState` to statyczne pola pod jedną blokadą (`Running`, `Cancel`, `Total`, `Done`, `Current`, `Error`, `FinishedAt`) plus dziennik do 100 wpisów. `TranscribeWorker.Start` wybiera klipy, ustawia stan i uruchamia zadanie, które dla każdego klipu: sprawdza anulowanie, pomija klipy w chmurze, usuwa z bazy klipy bez pliku, mierzy czas trwania i robi miniaturę, woła `Transcriber.Transcribe`, zapisuje segmenty w miejsce starych oraz `transcribed_at` i język. Błąd jednego klipu nie przerywa reszty; na końcu `Running=false` i `FinishedAt`.

8.4 Transcriber (Whisper)

1. **Audio:** `ffmpeg` dekoduje plik do mono 16 kHz s16le z filtrem `dynaudnorm=f=150:g=15` (wyrównanie głośności), limit 300 s; próbki są przeliczane na float.
2. **Dobór modelu** (`CandidateModels`): jeśli `KEEPCLIP_WHISPER_MODEL` — tylko on. Inaczej drabinka od pierwszego modelu, którego próg VRAM (`GpuInfo` przez `DXGI`) jest spełniony, z pominięciem modelu zablokowanego. Przed załadowaniem zapisuje `whisper_model_probe`; jeśli proces zginie w trakcie (brak pamięci GPU zabija proces bez wyjątku), następny start zobaczy znacznik i zablokuje ten model na stałe.
3. **Fabryka:** `WhisperFactory.FromPath`, model pobierany do `models/` przez `WhisperGgmlDownloader` z plikiem `.part` (przerwane pobieranie nie udaje gotowego modelu). Zapisuje w logu użyty silnik (Vulkan/CUDA/CPU).
4. **Przetwarzanie:** język z ustawień, beam search 5, bez kontekstu między segmentami, znaczniki czasu tokenów, próg braku mowy 0,85. Postęp 0,15–0,95 po czasie segmentu.
5. **Dzielenie:** segment Whispera jest dzielony po ciszy $\geq 1,5$ s między tokenami, ale tylko gdy zegary tokenów zgadzają się z segmentem (± 1 s); inaczej zostaje w całości.
6. **Schodzenie w dół:** wyjątek w trakcie (np. brak pamięci zgłoszony jako wyjątek) zwalnia fabrykę i próbuje lżejszego modelu; jawny wybór użytkownika nigdy nie jest podmieniany.

8.5 ClipService

Operacje na jednym klipie zwracające `Result<Dictionary>` (`IsSuccess`, `Code`, `Error`), które endpointy tłumaczą na HTTP: `Fix`, `Cut`, `DeleteAsync`, `Retranscribe` — opisane przy endpointach. Zależy od repozytoriów i `PlaybackProxyService` (unieważnianie kopii podglądowej).

8.6 PlaybackProxyService

Dla klipów powyżej 120 FPS trzyma zadania konwersji (`ConcurrentDictionary` po id klipu) i pamięć podręczną wyników `ffprobe`. Kopia: skalowanie do maks. 1920 px szerokości, 60 FPS CFR, najpierw NVENC (`h264_nvenc` z dekodowaniem CUDA), a gdy się nie uda, `libx264 veryfast crf 23`; audio AAC 160 kb/s; `+faststart`. Postęp czytany z `-progress pipe:1`. Pliki w `data/playback/<id>_<odcisk>.mp4`; stare odciski tego samego klipu są usuwane. Oryginał nigdy nie jest modyfikowany.

8.7 CloudService

- **UploadClipAsync**: wysyłka wznowialna z metadanymi `appProperties {keepclip:"1", game, duration, mtime}` do folderu `KeepClip`; potem `storage='cloud'`, `remote_id`, plik lokalny do Kosza (miniatura zostaje).
- **SyncFromCloudAsync** (pod semaforem): lista plików wideo z folderu; dopasowanie po `remote_id` (aktualizacja nazwy, rozmiaru, czasu), potem po nazwie dla rekordów chmurowych bez `remote_id`, w końcu import nowych z grą z `appProperties` lub wyciągniętą z nazwy pliku (wzorzec `Gra yyyy-MM-dd HH-mm-ss, cs2` → Counter-Strike 2, inaczej „Chmura”). Ścieżka docelowa jest unikalna (`_cloud_<8 znaków id>` przy kolizji).
- **DownloadClipAsync**: pobranie do `.part`, podmiana, `storage='local'`, kosz Google (błąd kosza nie cofa operacji).
- **ProxyVideoAsync**: przekazuje strumień z Dysku do odpowiedzi HTTP z nagłówkami zakresu.
- **TrashRemoteAsync**, **UploadFolderAsync**.

8.8 UpdateService, OnboardingService, SingleOperationProgress

`UpdateService` — opisany przy endpointach; jedna instalacja naraz (`Interlocked`). `OnboardingService` — czyta i zapisuje obiekt `onboarding` w ustawieniach pod blokadą; `CurrentVersion = 1`, a jego podbicie *nie* uruchamia samouczka ponownie u dotychczasowych użytkowników (od tego są podpowiedzi `seen_hints`). `SingleOperationProgress` — jeden zestaw pól statycznych opisujący bieżącą operację; `Report` ignoruje raporty ze starym identyfikatorem.

8.9 ReplayService (bufor powtórek) — skrót

Pełny opis potoku `ffmpeg` jest w dokumencie Infrastruktura. W skrócie: `ffmpeg` przechwytuje pulpit (`ddagrab` na GPU, `ddagrab + hwdownload` albo `gdiagrab`) i koduje H.264 (NVENC/AMF/QuickSync/x264) do 2-sekundowych segmentów MPEG-TS w `data/tmp/replay`. `AudioPump` miksuje wyjścia WASAPI i mikrofon do potoku `stdin`. Wątek-dozorca kasuje stare segmenty (zostaje długość powtórki + 2 segmenty). `Watchdog` restartuje `ffmpeg` po awarii (po 2 szybkich awariach schodzi na zgodniejszą metodę przechwytywania, po 6 zatrzymuje bufor). Zapis (`SaveAsync`) łączy segmenty bez ponownego kodowania obrazu (`-c:v copy`), a audio koduje ponownie, żeby zszyć segmenty; wynik łąduje w folderze wykrytej gry (nazwa procesu okna na pierwszym planie).

9. Wątki, blokady i stabilność

Wątek / zadanie	Kto tworzy	Ochrona
Kestrel (pula wątków)	ASP.NET Core	Wyjątek w endpointzie → 500; proces żyje.
KeepClip-UI (STA)	Program.cs	Application.ThreadException → CrashLog, aplikacja pracuje dalej.
KeepClip-Idle	StartIdleWatcher	Tylko odczyty; brak ryzyka wyjątku.
KeepClip-AudioPump	AudioPump.Start	Cała pętla w try/catch; awaria = ciche nagranie, wpis w logu.
KeepClip-ReplayWatchdog	StartWatchdog	Logika restartu w try/catch; wyjątek zatrzymuje tylko bufor i trafia do crash.log.
KeepClip-ReplayJanitor	StartJanitor	try/catch w każdej iteracji.
KeepClip-ShowSignal	ShellForm	try/catch.
Zadania Task.Run (miniatury, transkrypcja, proxy, aktualizacja, OAuth)	różne	Nieobserwowany wyjątek nie zamyka procesu; UnobservedTaskException loguje go do crash.log.

Blokady: TranscribeState.Lock (stan transkrypcji), ReplayService.Gate (start/stop bufora), Settings.Gate (zapis pliku), OnboardingService.Gate, CloudService.SyncGate (jedna synchronizacja naraz), OAuthService.TokenGate (odświeżanie tokenu), Scanner.ScanLock, Transcriber.ModelLock. Wszystkie są krótkie; blokad nie zagnieżdża się poza ReplayService (Gate → _stderrTail).

Nieodwracalność: operacje na plikach użytkownika zawsze idą przez Kosz (Trash.Send = SHFileOperation z FOF_ALLOWUNDO), nigdy przez File.Delete. Wyjątek: pliki tymczasowe aplikacji.

10. Jak rozszerzać backend

10.1 Nowy endpoint

1. Dodaj metodę w odpowiednim pliku `Endpoints/*.cs` (albo nowy plik z metodą `Map...Endpoints` i wywołaniem w `Program.cs`).
2. Ciało żądania jako `record` z polami `snake_case` na końcu pliku.
3. Walidację i błędy zwracaj przez `Api.Detail(kod, Strings.Get("klucz"), "kodBledu")`. Dodaj klucz do `Strings.cs` w czterech językach i `errors.kodBledu` do `frontend/i18n.js`, jeśli interfejs ma tłumaczyć.
4. Logikę umieść w `Services/`, dostęp do bazy w `Data/`.
5. Wywołaj `Heartbeat.Touch()`, jeśli endpoint jest efektem działania użytkownika.
6. Dodaj test w `tests/KeepClip.Tests/Api/` (wzorec: klasa w kolekcji `"api"`, reset bazy i ustawień w konstruktorze).

10.2 Nowe ustawienie

Czytaj przez `Settings.GetString/GetObject`, zapisuj przez `SetString/SetObject`. Nie dodawaj pamięci podręcznej — plik jest mały, a prostota chroni przed niespójnością między wątkami.

10.3 Nowa kolumna w bazie

Dopisz do schematu w `Db.Schema` oraz do `Migrate` jako `AddColumn`, bo `CREATE TABLE IF NOT EXISTS` nie zmienia istniejących baz.

10.4 Zasady

- Kod jest bez komentarzy; wiedza o „dlaczego” żyje w tych dokumentach (`docs/src/*.html`). Zmieniając zachowanie, zaktualizuj odpowiedni rozdział i przebuduj PDF (`node docs/build-pdf.js`).
- Nie wołaj `ffmpeg` ani `Google` w wątku UI. Długie operacje przez `Task.Run` z raportowaniem postępu.
- Nigdy nie kasuj plików użytkownika bez Kosza.
- Wersja aplikacji jest w dwóch miejscach: `KeepClip.csproj` i `installer/KeepClip.iss`.